استفاده از قفل بیت می‌تواند خطرناک باشد: قفل بیت ممکن است یک شود، ولی هرگز صفر نشود. اگر چنین حالتی پیش بیاید (برای مثال، به خاطر وجود یک باگ در سیستم‌عامل)، قاب قفل‌شده دیگر قابل استفاده نخواهد بود. برای نمونه، سولاریس قفل‌گذاری نوع «یادآوری[1]» را فراهم می‌کند، اما این اختیار را دارد که اگر انبار قاب آزاد زیادی ته بکشد یا اگر پردازه‌ی خاصی بخواهد صفحات زیادی را در حافظه قفل نماید این یادآوری‌ها را نادیده بگیرد.

۱۰-۱۰ مثال‌های سیستم‌عامل

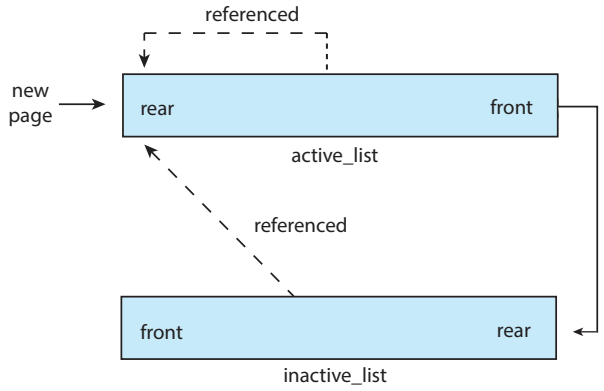
در این بخش، چگونگی مدیریت حافظه‌ی مجازی را در لینوکس، ویندوز و سولاریس بیان می‌کنیم.

۱۰-۱۰-۱ لینوکس

ما در بخش ۱۰-۸-۲، چگونگی مدیریت حافظه‌ی هسته با استفاده از تخصیص قالبی در لینوکس را بحث کردیم. حال، چگونگی مدیریت حافظه‌ی مجازی لینوکس را پوشش می‌دهیم. لینوکس با تخصیص صفحات از یک لیست قاب‌های آزاد از صفحه‌بندی نیازی استفاده می‌کند. به‌علاوه، آن از یک سیاست جایگزینی صفحه‌ی سراسری شبیه به الگوریتم ساعت تقریب LRU مطرح شده در بخش ۱۰-۴-۵-۲ استفاده می‌کند. لینوکس به‌منظور مدیریت حافظه، دو نوع لیست صفحه نگه می‌دارد: active_list و inactive_list. active_list شامل صفحاتی است که در حال استفاده محسوب می‌شوند، در حالی که inactive_list صفحاتی را شامل است که اخیراً مراجعه نداشته‌اند و برای پس‌گیری آمادگی دارند.

هر صفحه دارای یک بیت *دسترسی* است که هر بار که صفحه مراجعه می‌شود یک می‌گردد. (بیت‌های واقعی استفاده شده برای علامت‌زدن دسترسی صفحه بسته به معماری فرق می‌کنند.) موقعی که صفحه‌ای برای بار نخست تخصیص داده می‌شود، بیت دسترسی آن یک می‌شود، و به ته active_list افزوده می‌شود. با تشابهی، هر موقع که صفحه‌ای از active_list مراجعه می‌شود، بیت دسترسی آن یک شده و صفحه به ته لیست منتقل می‌شود. به‌صورت متناوب، بیت‌های دسترسی صفحات active_list صفر می‌شوند. به‌مرور زمان، صفحه‌ی کمتر اخیراً به‌کار رفته در جلوی active_list قرار خواهد گرفت. امکان دارد که از آنجا به ته inactive_list مهاجرت کند. اگر صفحه‌ای از inactive_list مراجعه شود، دوباره به ته active_list منتقل می‌شود. این الگو در شکل ۱۰-۲۹ نمایش داده می‌شود.

1. Hints



شکل ۱۰-۲۹ ساختارهای active_list و inactive_list لینوکس.

دو لیست به‌طور نسبی، متعادل نگه داشته می‌شوند، و موقعی که active_list بسیار بیشتر از inactive_list رشد پیدا می‌کند، صفحاتی از جلوی active_list به inactive_list منتقل می‌شوند که در آنجا برای پس‌گیری آمادگی دارند. هسته‌ی لینوکس، یک گماشته‌ی مبادله‌ی صفحه به بیرون، به‌نام پردازه‌ی kswapd دارند که به‌صورت متناوب، بیدار شده و میزان حافظه‌ی آزاد در سیستم را بررسی می‌کند. اگر حافظه‌ی آزاد، زیر یک آستانه‌ی معینی باشد، kswapd پویش صفحات را در inactive_list شروع کرده و صفحات آن‌ها را برای لیست آزاد پس می‌گیرد. مدیریت حافظه‌ی مجازی لینوکس با جزئیات بیشتر در فصل ۲۰ مورد بحث قرار می‌گیرد.

۱۰-۱۰-۲ ویندوز

ویندوز ۱۰ از سیستم‌های ۳۲ و ۶۴ بیتی اجرایی بر روی معماری‌های اینتل (IA-32 و x86-64) و ARM پشتیبانی می‌کند. در سیستم‌های ۳۲ بیتی، فضای آدرس مجازی پیش‌فرض یک پردازه ۲ گیگابایت است، هرچند که می‌تواند به ۳ گیگابایت گسترش یابد. سیستم‌های ۳۲ بیتی از حافظه‌ی فیزیکی ۴ گیگابایتی پشتیبانی می‌کنند. در سیستم‌های ۶۴ بیتی، ویندوز ۱۰ دارای یک فضای آدرس مجازی ۱۲۸ ترابایتی است و تا ۲۴ ترابایت حافظه‌ی فیزیکی پشتیبانی می‌کند. (نسخه‌های ویندوز سرور تا ۱۲۸ ترابایت حافظه‌ی فیزیکی را پشتیبانی می‌کنند.) ویندوز ۱۰ اکثر امکانات مدیریت حافظه را که تاکنون بیان شده‌اند، اعم از کتابخانه‌های مشترک، صفحه‌بندی نیازی، کپی-موقع-نوشتن، صفحه‌بندی، و فشرده‌سازی حافظه پیاده‌سازی می‌کند.

ویندوز ۱۰ حافظه‌ی مجازی را با استفاده از صفحه‌بندی نیازی با **خوشه‌بندی** پیاده‌سازی می‌کند، راهبردی که همسایگی مراجعات حافظه را تشخیص داده و لذا نه تنها با آوردن صفحه‌ی خطادار به حافظه به خطای صفحه رسیدگی می‌کند، بلکه چندین صفحه‌ای که بلافاصله قبل و بعد از صفحه‌ی خطادار هستند را نیز به داخل حافظه بارگذاری می‌کند. اندازه‌ی یک خوشه بسته به نوع صفحه فرق می‌کند. برای یک صفحه‌ی داده‌ای، یک خوشه شامل سه صفحه است (صفحه‌ی قبل و صفحه‌ی بعد از صفحه‌ی خطادار)؛ سایر خطاهای صفحه دارای خوشه‌هایی با اندازه‌ی هفت می‌باشند.

مؤلفه‌ی کلیدی مدیریت حافظه‌ی مجازی در ویندوز ۱۰ مدیریت مجموعه‌ی کاری است. موقعی که پردازه‌ای ایجاد می‌شود، یک مجموعه‌ی کاری کمینه‌ی ۵۰ صفحه‌ای و یک مجموعه‌ی کاری بیشینه با ۳۴۵ صفحه به آن گمارده می‌شود. **کمینه‌ی مجموعه‌ی کاری** تعداد حداقل صفحاتی است که برای پردازه تضمین است در حافظه خواهد داشت؛ اگر حافظه‌ی کافی فراهم باشد، پردازه می‌تواند صفحات بیشتری تا **بیشینه‌ی مجموعه‌ی کاریش** در اختیار بگیرد. مگر این که یک پردازه با **محدودیت‌های سخت مجموعه‌ی کاری** پیکربندی شود، این مقادیر ممکن است نادیده گرفته شود. یک پردازه می‌تواند فراتر از بیشینه‌ی مجموعه‌ی کاریش رشد پیدا کند در صورتی که حافظه‌ی کافی در دسترس باشد. با تشابهی، میزان حافظه‌ی تخصیص یافته به یک پردازه می‌تواند در دوره‌هایی که تقاضای حافظه بالاست به زیر کمینه‌ی آن تقلیل یابد.

ویندوز از الگوریتم ساعت تقریب LRU، همان‌گونه که در بخش ۱۰-۴-۵-۲ بیان شد، با ترکیبی از سیاست‌های جایگزینی صفحه‌ی محلی و سراسری استفاده می‌کند. مدیر حافظه‌ی مجازی لیستی از قاب‌صفحات آزاد نگه می‌دارد. مرتبط با این لیست، یک مقدار آستانه است که نشان می‌دهد آیا حافظه‌ی آزاد به‌قدر کافی در دسترس هست یا خیر. اگر یک خطای صفحه برای پردازه‌ای اتفاق بیفتد که زیر بیشینه‌ی مجموعه‌ی کاریش می‌باشد، مدیر حافظه‌ی مجازی یک صفحه از لیست صفحات آزاد تخصیص می‌دهد. اگر پردازه‌ای که در بیشینه‌ی مجموعه‌ی کاریش قرار دارد، گرفتار یک خطای صفحه شود و حافظه‌ی کافی در دسترس باشد، پردازه یک صفحه‌ی آزاد در اختیار می‌گیرد که به آن اجازه می‌دهد تا فراتر از بیشینه‌ی مجموعه‌ی کاریش رشد پیدا کند. اما، اگر میزان حافظه‌ی آزاد ناکافی باشد، هسته بایستی یک صفحه‌ای را از مجموعه‌ی کاری پردازه برای جایگزینی با استفاده از سیاست جایگزینی صفحه‌ی LRU محلی انتخاب کند.

موقعی که میزان حافظه‌ی آزاد به زیر آستانه می‌افتد، مدیر حافظه‌ی مجازی از یک تاکتیک جایگزینی سراسری تحت عنوان **پیرایش خودکار مجموعه‌ی کاری**[1] برای بازیابی مقدار به یک سطح بالای آستانه استفاده می‌کند. پیرایش خودکار مجموعه‌ی کاری با ارزیابی تعداد صفحات در اختیار پردازه‌ها کار می‌کند. اگر پردازه‌ای صفحات بیشتری از کمینه‌ی مجموعه‌ی کاریش در اختیار داشت، مدیر حافظه‌ی مجازی صفحاتی را از مجموعه‌ی کاری بر می‌دارد تا این که حافظه‌ی کافی در دسترس قرار بگیرد و یا پردازه به کمینه‌ی مجموعه‌ی کاریش برسد. پردازه‌های بزرگ‌تری که بیکار شده‌اند قبل از پردازه‌های فعال کوچک‌تر هدف قرار می‌گیرند. روال پیرایش ادامه می‌یابد تا این که حافظه‌ی آزاد کافی فراهم شود، حتی اگر لازم باشد صفحاتی از یک پردازه برداشته شوند که آن را زیر کمینه‌ی مجموعه‌ی کاریش قرار می‌دهد. ویندوز، پیرایش مجموعه‌ی کاری را برای هر دوی پردازه‌های مد کاربر و سیستمی انجام می‌دهد.

۱۰-۱۰-۳ سولاریس

زمانی که یک نخ در سولاریس گرفتار خطای صفحه می‌شود، هسته یک صفحه را از لیست صفحات آزادی که نگه می‌دارد به آن نخ تخصیص می‌دهد. بنابراین، ضرورت دارد که هسته،

1. Automatic working-set trimming

حجم کافی حافظه‌ی آزاد را در دسترس نگه بدارد. مرتبط با این لیست صفحات آزاد، پارامتری—به نام *lotsfree*—وجود دارد که آستانه‌ای را برای شروع صفحه‌بندی نشان می‌دهد. پارامتر *lotsfree* نوعاً برابر 1/64 اندازه‌ی حافظه‌ی فیزیکی موجود انتخاب می‌شود. در هر ثانیه، هسته چهار بار بررسی می‌کند که آیا حجم حافظه‌ی آزاد موجود کمتر از *lotsfree* می‌باشد یا خیر. اگر تعداد صفحات آزاد پایین‌تر از *lotsfree* باشد، پردازه‌ای به نام **pageout** کار خود را شروع می‌کند. پردازه‌ی pageout شبیه به الگوریتم شانس دوم که در بخش ۱۰-۴-۵-۲ مطرح شد عمل می‌کند، جز این که، این پردازه در پویش صفحات به جای استفاده از یک عقربه از دو عقربه استفاده می‌کند.

پردازه‌ی pageout به این صورت کار می‌کند: عقربه‌ی جلویی ساعت، تمامی صفحات داخل حافظه را پویش کرده و بیت مراجعه‌ی آنها را ۰ می‌کند. کمی بعد، عقربه‌ی عقبی ساعت، بیت مراجعه‌ی صفحات داخل حافظه را بررسی کرده و آن دسته از صفحاتی را که همچنان بیت آنها ۰ است به لیست آزاد اضافه می‌نماید و اگر محتوای آن‌ها تغییر یافته باشد آنها را در حافظه‌ی ثانویه می‌نویسد. سولاریس، خطاهای صفحه‌ی فرعی را با اجازه‌دادن به یک پردازه تا صفحه‌ای را از لیست آزاد پس بگیرد اگر صفحه قبل از این که دومرتبه به پردازه‌ی دیگر گمارده شود دست‌یابی شود، مدیریت می‌کند.

الگوریتم pageout برای کنترل نرخ پویش صفحات (معروف به *نرخ پویش*[1]) از پارامترهای متعددی استفاده می‌کند. نرخ پویش با واحد صفحه در ثانیه بیان می‌شود و بین *پویش کند* تا *پویش تند* تغییر می‌کند. زمانی که حافظه‌ی آزاد پایین‌تر از *lotsfree* بیفتد، پویش با نرخ *slowscan* صفحه در ثانیه شروع می‌شود و بسته به حجم حافظه‌ی آزاد موجود می‌تواند تا نرخ *fastscan* پیش برود. مقدار پیش‌فرض نرخ *slowscan* ۱۰۰ صفحه در ثانیه است. *fastscan* نوعاً برابر *نصف تعداد کل صفحات فیزیکی* انتخاب می‌شود که حداکثر آن ۸۱۹۲ صفحه در ثانیه است. این رویه در شکل ۱۰-۳۰ نشان داده می‌شود (که *fastscan* برابر حداکثر آن انتخاب شده است).

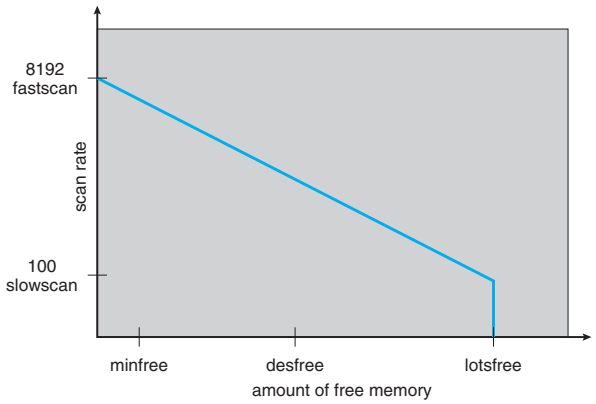
فاصله‌ی میان دو عقربه‌ی ساعت (با واحد صفحه) توسط یک پارامتر سیستم به نام *handspread* معین می‌شود. مدت زمانی که طول می‌کشد تا عقربه‌ی جلویی یک بیت را صفر کرده و عقربه‌ی عقبی مقدار آن را بررسی نماید به *scanrate* و *handspread* بستگی دارد. اگر *scanrate* برابر ۱۰۰ و *handspread* برابر ۱۰۲۴ صفحه باشد، حدود ۱۰ ثانیه سپری خواهد شد تا بیتی که عقربه‌ی جلویی یک کرده است توسط عقربه‌ی عقبی بررسی شود. با این وجود، به خاطر نیازهایی که برای سیستم حافظه وجود دارد، *نرخ پویش* چند هزارتایی رواج بیشتری دارد. این بدان معنی است که اغلب، مدت زمان بین پاک‌سازی بیت و بررسی آن بیشتر از چند ثانیه طول نمی‌کشد.

همان‌گونه که در بالا متذکر شدیم، پردازه‌ی pageout حافظه را در هر ثانیه، چهار بار بررسی می‌کند. با این وجود، اگر حافظه‌ی آزاد زیر مقدار *desfree* بیفتد (میزان مطلوب حافظه‌ی آزاد سیستم)، pageout در هر ثانیه صد مرتبه اجرا خواهد شد تا حافظه‌ی آزاد موجود را حداقل در حد *desfree* نگه بدارد (شکل ۱۰-۳۰). اگر پردازه‌ی pageout نتواند حجم حافظه‌ی آزاد را با گذشت

1. Scanrate

متوسط زمان ۳۰ ثانیه‌ای، در حد *desfree* نگه بدارد، هسته شروع به مبادله‌ی پردازه‌ها خواهد نمود و با این کار، صفحات تخصیصی پردازه‌ها را آزاد خواهد کرد. در حالت کلی، هسته دنبال پردازه‌هایی خواهد گشت که برای دوره‌های طولانی بیکار بوده‌اند. اگر سیستم قادر نباشد که حجم آزاد حافظه را در حد *minfree* حفظ کند، پردازه‌ی pageout با هر بار درخواست صفحه‌ی جدید فراخوانی می‌شود.

الگوریتم پویش صفحه از صفحات متعلق به کتابخانه‌هایی می‌گذرد که توسط چندین پردازه به‌اشتراک گذاشته می‌شوند، حتی اگر آن‌ها برای پس‌گیری توسط پویش‌گر آماده باشند. همین‌طور، الگوریتم بین صفحاتی که به پردازه‌ها تخصیص یافته‌اند و صفحاتی که به فایل‌های داده‌ای معمولی تخصیص پیدا کرده‌اند تفاوت قایل می‌شود. این موضوع به **صفحه‌بندی الویتی** معروف است و در بخش ۱۴-۶-۲ پوشش داده می‌شود.



شکل ۱۰-۳۰ پویشگر صفحه در سولاریس.

۱۰-۱۱ خلاصه

- حافظه‌ی مجازی، حافظه‌ی فیزیکی را به‌شکل یک آرایه‌ی ذخیره‌سازی یک‌شکل بی‌نهایت بزرگی انتزاعی می‌سازد.
- منافع حافظه‌ی مجازی عبارتند از: (۱) یک برنامه می‌تواند بزرگ‌تر از حافظه‌ی فیزیکی باشد، (۲) یک برنامه نیازی ندارد که به‌صورت کامل در داخل حافظه قرار بگیرید، (۳) پردازه‌ها می‌توانند حافظه را به‌اشتراک بگذارند، و (۴) پردازه‌ها می‌توانند مؤثرتر ایجاد شوند.
- صفحه‌بندی نیازی، تکنیکی است که با آن صفحات فقط زمانی بارگذاری می‌شوند که در طی اجرای برنامه تقاضا شوند. صفحاتی که هرگز تقاضا نمی‌شوند هرگز به داخل حافظه بارگذاری نمی‌شوند.
- خطای صفحه زمانی اتفاق می‌افتد که یک صفحه‌ای که فعلاً در حافظه نیست دسترسی می‌شود. بایستی صفحه از ذخیره‌ی پشتیبان به یک قاب موجود در داخل حافظه آورده شود.
- کپی-موقع-نوشتن به یک پردازه‌ی فرزند اجازه می‌دهد تا فضای آدرس یکسانی را

مثل والدش به‌اشتراک بگیرد. اگر پردازه‌ی فرزند و یا والد، صفحه‌ای را بنویسند (تغییر دهند)، یکی کپی از صفحه گرفته می‌شود.

- زمانی که حافظه‌ی موجود ته می‌کشد، یک الگوریتم جایگزینی صفحه، صفحه‌ی موجودی را از داخل حافظه با یک صفحه‌ی جدیدی جایگزین می‌کند. الگوریتم‌های جایگزینی صفحه عبارتند از FIFO، بهینه، و LRU. پیاده‌سازی الگوریتم‌های LRU محض غیرعملی است، و اکثر سیستم‌ها در عوض، از الگوریتم‌های تقریب LRU استفاده می‌کنند.
- الگوریتم‌های جایگزینی صفحه‌ی سراسری، یک صفحه از هر کدام از پردازه‌ها در سیستم که شد برای جایگزینی انتخاب می‌کنند، در حالی که الگوریتم‌های جایگزینی صفحه‌ی محلی، صفحه را برای جایگزینی از پردازه‌ی خطادار انتخاب می‌کنند.
- کوبیدگی زمانی اتفاق می‌افتد که یک سیستم، بیشتر وقت خود را به‌جای اجرا صرف صفحه‌بندی می‌نماید.
- همسایگی معرف یک مجموعه از صفحات است که به‌صورت فعال با همدیگر استفاده می‌شوند. همچنان‌که یک پردازه اجرا می‌شود، از یک همسایگی به همسایگی دیگر جابجا می‌شود. مجموعه‌ی کاری بر اساس همسایگی است و به‌صورت مجموعه‌ای از صفحاتی که فعلاً توسط یک پردازه استفاده می‌شوند تعریف می‌شود.
- فشرده‌سازی حافظه یک تکنیک مدیریت حافظه است که تعدادی از صفحات را به‌شکل یک تک‌صفحه فشرده می‌سازد. حافظه‌ی فشرده شده یک جایگزینی برای صفحه‌بندی است و در سیستم‌های موبایلی که از صفحه‌بندی پشتیبانی نمی‌کنند استفاده می‌شود.
- حافظه‌ی هسته متفاوت از پردازه‌های مد کاربر تخصیص داده می‌شود؛ آن به‌صورت قالب‌های همجواری از اندازه‌های متغیر تخصیص داده می‌شود. دو تکنیک رایج برای تخصیص حافظه‌ی هسته عبارتند از: (۱) سیستم نیمه و (۲) تخصیص قالبی.
- حجم پوششی TLB به میزان حافظه‌ای مربوط می‌شود که از TLB قابل دست‌یابی است و برابر تعداد وارده‌های TLB ضرب در اندازه‌ی صفحه می‌باشد. یک تکنیک برای افزایش حجم پوششی TLB، افزایش اندازه‌ی صفحات است.
- لینوکس، ویندوز، و سولاریس، حافظه‌ی مجازی را شبیه به هم با استفاده از صفحه‌بندی نیازی و کپی–موقع–نوشتن در میان سایر امکانات، مدیریت می‌کنند. هر سیستم از یک گونه‌ای از تقریب LRU نیز تحت عنوان الگوریتم ساعت استفاده می‌کنند.

خودآزمایی

۱۰-۱ خطاهای صفحه تحت چه شرایطی پیش می‌آیند؟ عملیاتی را که سیستم‌عامل با وقوع خطای صفحه در پیش می‌گیرد بیان کنید.

۱۰-۲ فرض کنید که رشته‌ی مراجعه‌ی صفحه‌ی یک پردازه را با m قاب (که همگی در ابتدا خالی هستند) در اختیار دارید. رشته‌ی مراجعه به صفحه به طول p و n